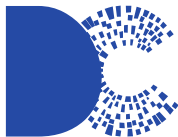




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Engenharia reversa

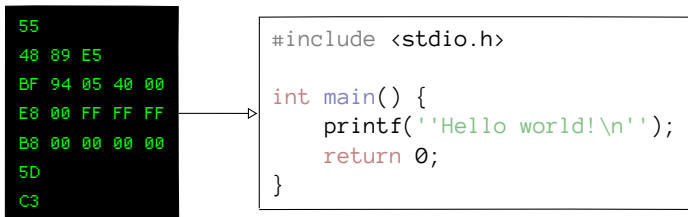
Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

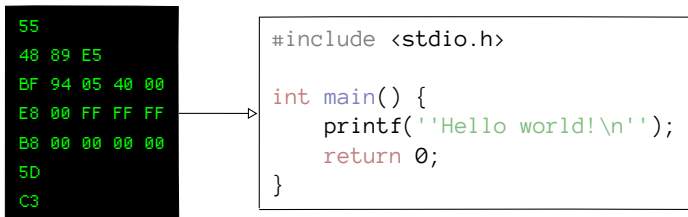
Introdução

- ▶ O que é engenharia reversa?
 - ▶ É o processo para entender o comportamento de um sistema a partir de sua implementação



Introdução

- ▶ O que é engenharia reversa?
 - ▶ É o processo para entender o comportamento de um sistema a partir de sua implementação



Código binário → Estrutura
Função
Projeto

Introdução

- ▶ Qual conhecimento é necessário?

Introdução

- ▶ Qual conhecimento é necessário?
 - ▶ Especificação da arquitetura de hardware
 - ▶ Instruções e registradores
 - ▶ Convenções da ABI
 - ▶ Mecanismos de endereçamento e interrupção

Introdução

- ▶ Qual conhecimento é necessário?
 - ▶ Especificação da arquitetura de hardware
 - ▶ Instruções e registradores
 - ▶ Convenções da ABI
 - ▶ Mecanismos de endereçamento e interrupção
 - ▶ Infraestrutura de software
 - ▶ Linguagens de programação
 - ▶ Ferramentas de desenvolvimento
 - ▶ Conceitos de sistema operacional
 - ▶ Extração de dados e desmontagem do código
 - ▶ Análise dinâmica do comportamento (depuração)

Introdução

- ▶ Qual conhecimento é necessário?
 - ▶ Especificação da arquitetura de hardware
 - ▶ Instruções e registradores
 - ▶ Convenções da ABI
 - ▶ Mecanismos de endereçamento e interrupção
 - ▶ Infraestrutura de software
 - ▶ Linguagens de programação
 - ▶ Ferramentas de desenvolvimento
 - ▶ Conceitos de sistema operacional
 - ▶ Extração de dados e desmontagem do código
 - ▶ Análise dinâmica do comportamento (depuração)
 - ▶ Técnicas de ofuscação de código
 - ▶ Dados da aplicação
 - ▶ Fluxo de controle
 - ▶ Segurança por obscuridade

Introdução

- ▶ Qual conhecimento é necessário?
 - ▶ Especificação da arquitetura de hardware
 - ▶ Instruções e registradores
 - ▶ Convenções da ABI
 - ▶ Mecanismos de endereçamento e interrupção
 - ▶ Infraestrutura de software
 - ▶ Linguagens de programação
 - ▶ Ferramentas de desenvolvimento
 - ▶ Conceitos de sistema operacional
 - ▶ Extração de dados e desmontagem do código
 - ▶ Análise dinâmica do comportamento (depuração)
 - ▶ Técnicas de ofuscação de código
 - ▶ Dados da aplicação
 - ▶ Fluxo de controle
 - ▶ Segurança por obscuridade

Conhecimento + Paciência + Tempo = Sucesso?

Introdução

- ▶ Qual conhecimento é necessário?
 - ▶ Especificação da arquitetura de hardware
 - ▶ Instruções e registradores
 - ▶ Convenções da ABI
 - ▶ Mecanismos de endereçamento e interrupção
 - ▶ Infraestrutura de software
 - ▶ Linguagens de programação
 - ▶ Ferramentas de desenvolvimento
 - ▶ Conceitos de sistema operacional
 - ▶ Extração de dados e desmontagem do código
 - ▶ Análise dinâmica do comportamento (depuração)
 - ▶ Técnicas de ofuscação de código
 - ▶ Dados da aplicação
 - ▶ Fluxo de controle
 - ▶ Segurança por obscuridade

O fracasso é sempre uma possibilidade...

Introdução

- ▶ Qual conhecimento é necessário?
 - ▶ Especificação da arquitetura de hardware
 - ▶ Instruções e registradores
 - ▶ Convenções da ABI
 - ▶ Mecanismos de endereçamento e interrupção
 - ▶ Infraestrutura de software
 - ▶ Linguagens de programação
 - ▶ Ferramentas de desenvolvimento
 - ▶ Conceitos de sistema operacional
 - ▶ Extração de dados e desmontagem do código
 - ▶ Análise dinâmica do comportamento (depuração)
 - ▶ Técnicas de ofuscação de código
 - ▶ Dados da aplicação
 - ▶ Fluxo de controle
 - ▶ Segurança por obscuridade

while(!sucesso) tente_novamente();

Introdução

- ▶ Por que fazer engenharia reversa?

Introdução

- ▶ Por que fazer engenharia reversa?
 - ▶ Auditoria de software
 - ▶ Privacidade e segurança da informação
 - ▶ Violações de direito autoral ou de patente

Introdução

- ▶ Por que fazer engenharia reversa?
 - ▶ Auditoria de software
 - ▶ Privacidade e segurança da informação
 - ▶ Violações de direito autoral ou de patente
 - ▶ Criar versões compatíveis de produtos comerciais
 - ▶ Sem documentação disponível pelo fabricante
 - ▶ Abordagem *clean room* para evitar infrações de direito autoral ou de propriedade intelectual
 - ▶ Redução de custos e suporte para outras plataformas

Introdução

- ▶ Por que fazer engenharia reversa?
 - ▶ Auditoria de software
 - ▶ Privacidade e segurança da informação
 - ▶ Violações de direito autoral ou de patente
 - ▶ Criar versões compatíveis de produtos comerciais
 - ▶ Sem documentação disponível pelo fabricante
 - ▶ Abordagem *clean room* para evitar infrações de direito autoral ou de propriedade intelectual
 - ▶ Redução de custos e suporte para outras plataformas
 - ▶ Propósito educacional
 - ▶ Ensino e pesquisa
 - ▶ Satisfação pessoal

Introdução

- ▶ Questões éticas e legais

Introdução

- ▶ Questões éticas e legais
 - ▶ *Black, grey ou white hat*
 - ▶ Intenção de uso $\times$ permissão de acesso
 - ▶ Hacktivismo

Introdução

- ▶ Questões éticas e legais
 - ▶ *Black, grey* ou *white hat*
 - ▶ Intenção de uso $\times$ permissão de acesso
 - ▶ Hacktivismo
 - ▶ Informação proprietária
 - ▶ Proteção do código fonte
 - ▶ Contrato de uso (EULA)

Introdução

- ▶ Questões éticas e legais
 - ▶ *Black, grey* ou *white hat*
 - ▶ Intenção de uso $\times$ permissão de acesso
 - ▶ Hacktivismo
 - ▶ Informação proprietária
 - ▶ Proteção do código fonte
 - ▶ Contrato de uso (EULA)
 - ▶ Limites da proteção de software
 - ▶ Cópia legítima do programa
 - ▶ Correção de erro ou utilização pessoal
 - ▶ Garantia de interoperabilidade

Engenharia reversa

► Diretivas de compilação

```
$ gcc -Wall -O3 mysterious.c -o mysterious.not_stripped
```

Com informações de símbolos (*not stripped*)

Engenharia reversa

► Diretivas de compilação

```
$ gcc -Wall -O3 mysterious.c -o mysterious.not_stripped  
$ gcc -Wall -O3 -s mysterious.c -o mysterious.stripped
```

Sem informações de símbolos (*stripped*)

Engenharia reversa

► Diretivas de compilação

```
$ gcc -Wall -O3 mysterious.c -o mysterious.not_stripped  
$ gcc -Wall -O3 -s mysterious.c -o mysterious.stripped  
$ gcc -Wall -O3 -static -s mysterious.c -o mysterious.stripped_static
```

Sem informação de símbolos e linkagem estática

Engenharia reversa

► Diretivas de compilação

```
$ gcc -Wall -O3 mysterious.c -o mysterious.not_stripped
$ gcc -Wall -O3 -s mysterious.c -o mysterious.stripped
$ gcc -Wall -O3 -static -s mysterious.c -o mysterious.stripped_static
$ ls -lha mysterious.*
-rw-r--r-- 1 bruno bruno 73 jul 17 09:45 mysterious.c
-rwxr-xr-x 1 bruno bruno 8,3K jul 17 09:45 mysterious.mac
-rwxr-xr-x 1 bruno bruno 17K jul 17 10:06 mysterious.not_stripped
-rwxr-xr-x 1 bruno bruno 15K jul 17 10:06 mysterious.stripped
-rwxr-xr-x 1 bruno bruno 675K jul 17 10:06 mysterious.stripped_static
$
```

Comparativo de tamanho dos binários gerados

Engenharia reversa

► Tipo do arquivo (*file*)

```
$ file mysterious.mac  
mysterious.mac: Mach-O 64-bit x86_64 executable, flags:<NOUNDEFS|DYLDL  
INK|TWOLEVEL|PIE>
```

Engenharia reversa

► Tipo do arquivo (*file*)

```
$ file mysterious.mac
mysterious.mac: Mach-O 64-bit x86_64 executable, flags:<NOUNDEFS|DYLDL
INK|TWOLEVEL|PIE>
$ file mysterious.not_stripped
mysterious.not_stripped: ELF 64-bit LSB pie executable, x86-64, versio
n 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.
2, BuildID[sha1]=de440972b7500f4dadc7c8aaf18f14d445d0e17f, for GNU/Li
nux 3.2.0, not stripped
```

Engenharia reversa

► Tipo do arquivo (*file*)

```
$ file mysterious.mac
mysterious.mac: Mach-O 64-bit x86_64 executable, flags:<NOUNDEFS|DYLDL
INK|TWOLEVEL|PIE>
$ file mysterious.not_stripped
mysterious.not_stripped: ELF 64-bit LSB pie executable, x86-64, versio
n 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.
2, BuildID[sha1]=de440972b750f4dad7c8aaf18f14d445d0e17f, for GNU/Li
nux 3.2.0, not stripped
$ file mysterious.stripped
mysterious.stripped: ELF 64-bit LSB pie executable, x86-64, version 1
(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, Buil
dID[sha1]=a07b5c7f0d6629a1bd82d1f6ea380c35713bb0ce, for GNU/Linux
3.2.0, stripped
```

Engenharia reversa

► Tipo do arquivo (*file*)

```
$ file mysterious.mac
mysterious.mac: Mach-O 64-bit x86_64 executable, flags:<NOUNDEFS|DYLDL
INK|TWOLEVEL|PIE>
$ file mysterious.not_stripped
mysterious.not_stripped: ELF 64-bit LSB pie executable, x86-64, versio
n 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.
2, BuildID[sha1]=de440972b7500f4dadc7c8aaf18f14d445d0e17f, for GNU/Li
nux 3.2.0, not stripped
$ file mysterious.stripped
mysterious.stripped: ELF 64-bit LSB pie executable, x86-64, version 1
(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, Buil
dID[sha1]=a07b5c7f0d6629a1bd82d1f6ea380c35713bb0ce, for GNU/Linux
3.2.0, stripped
$ file mysterious.stripped_static
mysterious.stripped_static: ELF 64-bit LSB executable, x86-64, version
1 (GNU/Linux), statically linked, BuildID[sha1]=45a3ef3df72b8c55faa1ef
d94dd4f154a67f30e7, for GNU/Linux 3.2.0, stripped
$
```

Engenharia reversa

► Símbolos do arquivo (*nm*)

```
$ nm mysterious.not_stripped
0000000000004030 B __bss_start
0000000000004030 b completed.7287
                w __cxa_finalize@@GLIBC_2.2.5
0000000000004020 D __data_start
0000000000004020 W data_start
0000000000001090 t deregister_tm_clones
...
0000000000001160 T __libc_csu_init
                U __libc_start_main@@GLIBC_2.2.5
0000000000001040 T main
                U puts@@GLIBC_2.2.5
00000000000010c0 t register_tm_clones
0000000000001060 T _start
0000000000004030 D __TMC_END__
```

Engenharia reversa

► Símbolos do arquivo (*nm*)

```
$ nm mysterious.not_stripped
0000000000004030 B __bss_start
0000000000004030 b completed.7287
                w __cxa_finalize@@GLIBC_2.2.5
0000000000004020 D __data_start
0000000000004020 W data_start
0000000000001090 t deregister_tm_clones
...
0000000000001160 T __libc_csu_init
                U __libc_start_main@@GLIBC_2.2.5
0000000000001040 T main
                U puts@@GLIBC_2.2.5
00000000000010c0 t register_tm_clones
0000000000001060 T _start
0000000000004030 D __TMC_END__
$ nm mysterious.stripped
nm: mysterious.stripped: no symbols
$
```

Engenharia reversa

► Cadeias de caracteres do arquivo (*strings*)

```
$ strings mysterious.not_stripped
/lib64/ld-linux-x86-64.so.2
...
Hello world!
...
__bss_start
main
...
.data
.bss
.comment
```

Engenharia reversa

► Cadeias de caracteres do arquivo (*strings*)

```
$ strings mysterious.not_stripped
/lib64/ld-linux-x86-64.so.2
...
Hello world!
...
__bss_start
main
...
.data
.bss
.comment
$ strings mysterious.stripped
/lib64/ld-linux-x86-64.so.2
...
Hello world!
...
.data
.bss
.comment
```

Engenharia reversa

► Cadeias de caracteres do arquivo (*strings*)

```
$ strings mysterious.stripped
/lib64/ld-linux-x86-64.so.2
...
Hello world!
...
.data
.bss
.comment
$ strings mysterious.mac
__PAGEZERO
...
Hello world!
...
!main
__mh_execute_header
_main
_puts
dyld_stub_binder
$
```

Engenharia reversa

- ▶ Desmontagem completa do código (*objdump*)
 - ▶ Código da função *main*

```
$ objdump -Dz -M intel mysterious.stripped
...
00000000000001040 <.text>:
1040: 48 83 ec 08          sub rsp,0x8
1044: 48 8d 3d b9 0f 00 00 lea rdi,[rip+0xfb9] # 2004 <puts@plt+0xfd4>
104b: e8 e0 ff ff ff      call 1030 <puts@plt>
1050: 31 c0               xor eax,eax
1052: 48 83 c4 08          add rsp,0x8
1056: c3                 ret
...
```

Engenharia reversa

- ▶ Desmontagem completa do código (*objdump*)
 - ▶ Código da função *main*

```
$ objdump -Dz -M intel mysterious.stripped
...
00000000000001040 <.text>:
1040: 48 83 ec 08          sub rsp,0x8
1044: 48 8d 3d b9 0f 00 00 lea rdi,[rip+0xfb9] # 2004 <puts@plt+0xfd4>
104b: e8 e0 ff ff         call 1030 <puts@plt>
1050: 31 c0               xor eax,eax
1052: 48 83 c4 08          add rsp,0x8
1056: c3                 ret
...
```

Chamada para função *puts*
para impressão de caracteres no terminal

Engenharia reversa

► Análise em tempo de execução

```
$ gdb ./mysterious.stripped
GNU gdb (GDB) 8.3
...
Reading symbols from ./mysterious.stripped...
(No debugging symbols found in ./mysterious.stripped)
```

Engenharia reversa

► Análise em tempo de execução

```
$ gdb ./mysterious.stripped
GNU gdb (GDB) 8.3
...
Reading symbols from ./mysterious.stripped...
(No debugging symbols found in ./mysterious.stripped)
(gdb) starti
...
0x00007ffff7fd4000 in _start () from /lib64/ld-linux-x86-64.so.2
```

Engenharia reversa

► Análise em tempo de execução

```
$ gdb ./mysterious.stripped
GNU gdb (GDB) 8.3
...
Reading symbols from ./mysterious.stripped...
(No debugging symbols found in ./mysterious.stripped)
(gdb) starti
...
0x00007ffff7fd4000 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) break puts
Breakpoint 1 at 0x55555555030
(gdb) continue
Continuing.
```

Engenharia reversa

► Análise em tempo de execução

```
$ gdb ./mysterious.stripped
GNU gdb (GDB) 8.3
...
Reading symbols from ./mysterious.stripped...
(No debugging symbols found in ./mysterious.stripped)
(gdb) start
...
0x00007ffff7fd4000 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) break puts
Breakpoint 1 at 0x55555555030
(gdb) continue
Continuing.

Breakpoint 1, 0x00007ffff7e3cee0 in puts () from /usr/lib/libc.so.6
(gdb) print/x $rdi
$1 = 0x555555556004
(gdb) print (char*)(0x555555556004)
$2 = 0x555555556004 "Hello world!"
```

Engenharia reversa

► Análise em tempo de execução

```
GNU gdb (GDB) 8.3
...
Reading symbols from ./mysterious.stripped...
(No debugging symbols found in ./mysterious.stripped)
(gdb) start
...
0x00007ffff7fd4000 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) break puts
Breakpoint 1 at 0x55555555030
(gdb) continue
Continuing.

Breakpoint 1, 0x00007ffff7e3cee0 in puts () from /usr/lib/libc.so.6
(gdb) print/x $rdi
$1 = 0x555555556004
(gdb) print (char*)(0x555555556004)
$2 = 0x555555556004 "Hello world!"
(gdb) set { char[13] } 0x555555556004 = "H3110 w0416!"
(gdb) c
Continuing.
H3110 w0416!
[Inferior 1 (process 1805) exited normally]
```

Engenharia reversa

► Análise em tempo de execução

```
...
Reading symbols from ./mysterious.stripped...
(No debugging symbols found in ./mysterious.stripped)
(gdb) start
...
0x00007ffff7fd4000 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) break puts
Breakpoint 1 at 0x55555555030
(gdb) continue
Continuing.

Breakpoint 1, 0x00007ffff7e3cee0 in puts () from /usr/lib/libc.so.6
(gdb) print/x $rdi
$1 = 0x555555556004
(gdb) print (char*)(0x555555556004)
$2 = 0x555555556004 "Hello world!"
(gdb) set { char[13] } 0x555555556004 = "H3110 w0416!"
(gdb) c
Continuing.
H3110 w0416!
[Inferior 1 (process 1805) exited normally]
(gdb) quit
$
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
$ xxd -g 1 mysterious.stripped > mysterious.stripped.hex  
$ vim mysterious.stripped.hex
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
00001f70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001f80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001f90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fa0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001ff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00002000: 01 00 02 00 56 65 72 79 20 65 61 73 79 21 21 21 ....Hello world!
00002010: 00 00 00 00 01 1b 03 3b 30 00 00 00 05 00 00 00 .....;0.....
00002020: 0c f0 ff ff 64 00 00 00 2c f0 ff ff 8c 00 00 00 ....d...,.....
00002030: 4c f0 ff ff 4c 00 00 00 4c f1 ff ff a4 00 00 00 L...L...L.....
00002040: bc f1 ff ff ec 00 00 00 14 00 00 00 00 00 00 00 .....
00002050: 01 7a 52 00 01 78 10 01 1b 0c 07 08 90 01 00 00 .zR..x.....
00002060: 14 00 00 00 1c 00 00 00 f8 ef ff ff 2f 00 00 00 ...../...
00002070: 00 44 07 10 00 00 00 00 24 00 00 00 34 00 00 00 .D.....$....4...
00002080: a0 ef ff ff 20 00 00 00 00 0e 10 46 0e 18 4a 0f ....F..J.
00002090: 0b 77 08 00 00 3f 1a 3b 2a 33 24 22 00 00 00 00 .w...?.;*3$"....
000020a0: 14 00 00 00 5c 00 00 00 98 ef ff ff 17 00 00 00 ....\.....
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
00001f70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001f80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001f90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fa0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001fe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001ff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00002000: 01 00 02 00 53 6f 6f 6f 20 65 61 73 79 20 64 75 ....Sooo easy du
00002010: 64 65 21 00 01 1b 03 3b 30 00 00 00 05 00 00 00 del....;0.....
00002020: 0c f0 ff ff 64 00 00 00 2c f0 ff ff 8c 00 00 00 ....d....
00002030: 4c f0 ff ff 4c 00 00 00 4c f1 ff ff a4 00 00 00 L...L...L.....
00002040: bc f1 ff ff ec 00 00 00 14 00 00 00 00 00 00 00 .....
00002050: 01 7a 52 00 01 78 10 01 1b 0c 07 08 90 01 00 00 .zR...x.....
00002060: 14 00 00 00 1c 00 00 00 f8 ef ff ff 2f 00 00 00 ...../...
00002070: 00 44 07 10 00 00 00 00 24 00 00 00 34 00 00 00 .D.....$....4...
00002080: a0 ef ff ff 20 00 00 00 00 0e 10 46 0e 18 4a 0f ....F..J.
00002090: 0b 77 08 80 00 3f 1a 3b 2a 33 24 22 00 00 00 00 .w...?.*3$"....
000020a0: 14 00 00 00 5c 00 00 00 98 ef ff ff 17 00 00 00 ....\.....
:wq
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
$ xxd -g 1 mysterious.stripped > mysterious.stripped.hex  
$ vim mysterious.stripped.hex
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
$ xxd -g 1 mysterious.stripped > mysterious.stripped.hex  
$ vim mysterious.stripped.hex  
$ xxd -r mysterious.stripped.hex > mysterious.stripped.edited
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
$ xxd -g 1 mysterious.stripped > mysterious.stripped.hex  
$ vim mysterious.stripped.hex  
$ xxd -r mysterious.stripped.hex > mysterious.stripped.edited  
$ chmod +x mysterious.stripped.edited
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
$ xxd -g 1 mysterious.stripped > mysterious.stripped.hex
$ vim mysterious.stripped.hex
$ xxd -r mysterious.stripped.hex > mysterious.stripped.edited
$ chmod +x mysterious.stripped.edited
$ ./mysterious.stripped.edited
Sooo easy dude!
```

Engenharia reversa

- ▶ Análise estática
 - ▶ Editor binário

```
$ xxd -g 1 mysterious.stripped > mysterious.stripped.hex
$ vim mysterious.stripped.hex
$ xxd -r mysterious.stripped.hex > mysterious.stripped.edited
$ chmod +x mysterious.stripped.edited
$ ./mysterious.stripped.edited
Sooo easy dude!
$ exit
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
$ objdump -Dz ---section=.text -M intel mysterious.stripped > mysterious  
.stripped.text  
$ objdump -Dz ---section=.rodata -M intel mysterious.stripped > mysterio  
us.stripped.rodata  
...  
$ vim mysterious.s
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
.intel_syntax noprefix
.global main
.text

main:
    lea rdi, [rip + parameter]
    call puts
    xor rax, rax
    ret

.section .rodata
parameter:
    .string "Why so easy?"
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
.intel_syntax noprefix
.global main
.text

main:
    lea rdi, [rip + parameter]
    call puts
    xor rax, rax
    ret

.section .rodata
parameter:
    .string "Why so easy?"

:wq
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
$ objdump -Dz ---section=.text -M intel mysterious.stripped > mysterious  
.stripped.text  
$ objdump -Dz ---section=.rodata -M intel mysterious.stripped > mysterio  
us.stripped.rodata  
...  
$ vim mysterious.s
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
$ objdump -Dz ---section=.text -M intel mysterious.stripped > mysterious  
.stripped.text  
$ objdump -Dz ---section=.rodata -M intel mysterious.stripped > mysterio  
us.stripped.rodata  
...  
$ vim mysterious.s  
$ gcc -Wall -O3 mysterious.s -o mysterious.reversed
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
$ objdump -Dz ---section=.text -M intel mysterious.stripped > mysterious
.stripped.text
$ objdump -Dz ---section=.rodata -M intel mysterious.stripped > mysterio
us.stripped.rodata
...
$ vim mysterious.s
$ gcc -Wall -O3 mysterious.s -o mysterious.reversed
$ ./mysterious.reversed
Why so easy?
```

Engenharia reversa

- ▶ Análise estática
 - ▶ ELF → Código de montagem → ELF

```
$ objdump -Dz ---section=.text -M intel mysterious.stripped > mysterious
.stripped.text
$ objdump -Dz ---section=.rodata -M intel mysterious.stripped > mysterio
us.stripped.rodata
...
$ vim mysterious.s
$ gcc -Wall -O3 mysterious.s -o mysterious.reversed
$ ./mysterious.reversed
Why so easy?
$ exit
```

Exercício

- ▶ Replique os experimentos de engenharia reversa com técnicas dinâmicas e estáticas
 - ▶ Se fosse utilizado a versão binária sem informações de símbolos e com compilação estática, quais seriam as diferenças no processo de engenharia reversa?
 - ▶ Estude ferramentas alternativas de análise e edição, como *Radare2/Cutter* e *Hexedit*

Ofuscação

- ▶ O que é ofuscação de código?
 - ▶ É a aplicação de transformações que dificultam o processo de engenharia reversa, mas mantendo o comportamento original (equivalência semântica)

Ofuscação

- ▶ O que é ofuscação de código?
 - ▶ É a aplicação de transformações que dificultam o processo de engenharia reversa, mas mantendo o comportamento original (equivalência semântica)
- ▶ Por que deixar "menos fácil"?
 - ▶ Permitir que softwares maliciosos (*malware*) deixem de ser detectados por ferramentas de segurança
 - ▶ Proteger a propriedade intelectual para evitar a replicação não autorizada do seu funcionamento
 - ▶ Gerenciar os direitos autorais (DRM) de conteúdos digitais, como chaves de produtos, músicas ou filmes

Ofuscação

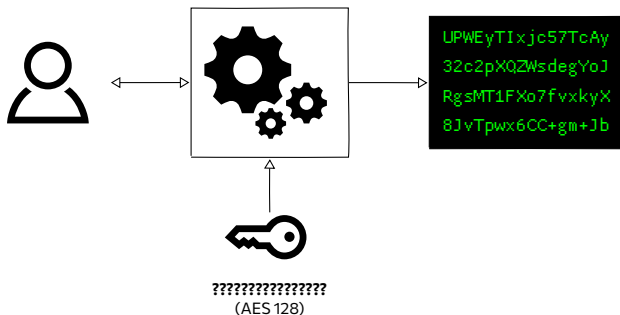
- ▶ Abordagem de caixa branca (*white box*)
 - ▶ É utilizada em sistemas criptográficos, onde todos os detalhes sobre o funcionamento são fornecidos, entretanto, sem expor as chaves privadas

Ofuscação

- ▶ Abordagem de caixa branca (*white box*)
 - ▶ É utilizada em sistemas criptográficos, onde todos os detalhes sobre o funcionamento são fornecidos, entretanto, sem expor as chaves privadas
 - ▶ A segurança consiste em demonstrar a robustez das técnicas empregadas, impedindo um ataque viável, mesmo quando o atacante possua todos os detalhes de implementação do sistema

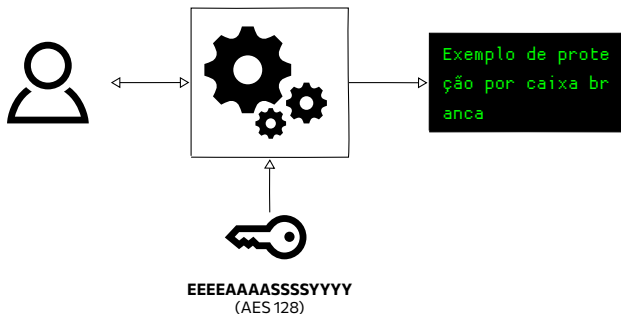
Ofuscação

- ▶ Abordagem de caixa branca (*white box*)
 - ▶ É utilizada em sistemas criptográficos, onde todos os detalhes sobre o funcionamento são fornecidos, entretanto, sem expor as chaves privadas
 - ▶ A segurança consiste em demonstrar a robustez das técnicas empregadas, impedindo um ataque viável, mesmo quando o atacante possua todos os detalhes de implementação do sistema



Ofuscação

- ▶ Abordagem de caixa branca (*white box*)
 - ▶ É utilizada em sistemas criptográficos, onde todos os detalhes sobre o funcionamento são fornecidos, entretanto, sem expor as chaves privadas
 - ▶ A segurança consiste em demonstrar a robustez das técnicas empregadas, impedindo um ataque viável, mesmo quando o atacante possua todos os detalhes de implementação do sistema

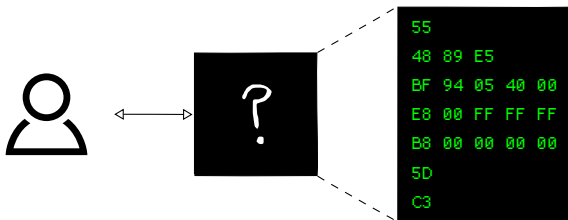


Ofuscação

- ▶ Abordagem de caixa preta (*black box*)
 - ▶ Consiste no desenvolvimento do código fonte com técnicas que visam dificultar o entendimento da implementação binária gerada pelo compilador

Ofuscação

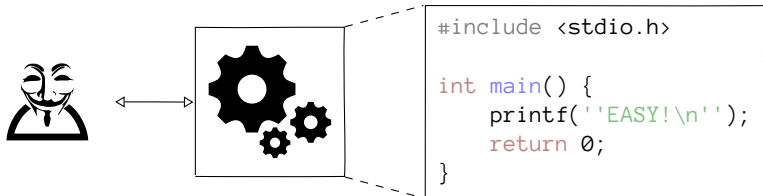
- ▶ Abordagem de caixa preta (*black box*)
 - ▶ Consiste no desenvolvimento do código fonte com técnicas que visam dificultar o entendimento da implementação binária gerada pelo compilador



Segurança por obscuridade

Ofuscação

- ▶ Abordagem de caixa preta (*black box*)
 - ▶ Consiste no desenvolvimento do código fonte com técnicas que visam dificultar o entendimento da implementação binária gerada pelo compilador



Não garante proteção do software,
mas pode dificultar bastante a sua análise

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Inserção de código morto (*dead code*): operações que não afetam o comportamento

```
1 // E/S padrão
2 #include <stdio.h>
3 ...
4
5 uint32_t __attribute__((noinline)) f(uint32_t n) {
6     if(n == 0) return 1;
7     else return n * f(n - 1);
8 }
9
10 uint32_t g(uint32_t* V, uint32_t n) {
11     uint32_t i, s = 0;
12     for(i = 0; i < n; i++) {
13         uint32_t j = i % 10;
14         s += V[i] + f(j) - (f(j + 1) / (j + 1));
15     }
16     return s / n;
17 }
18 ...
```

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Inserção de código morto (*dead code*): operações que não afetam o comportamento

```
1 // E/S padrão
2 #include <stdio.h>
3 ...
17 int main() {
18     uint32_t i, n = 1000000;
19     uint32_t* V = (uint32_t*)(malloc(n *
20         sizeof(uint32_t)));
21     for(i = 0; i < n; i++)
22         V[i] = rand();
23     printf("Average = %u\n", g(V, n));
24     return 0;
25 }
```

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Inserção de código morto (*dead code*): operações que não afetam o comportamento

```
$ gcc -Wall -O3 deadcode.c -o deadcode.elf
$ objdump -Dz -M intel deadcode.elf
...
0000000000001060 <main>:
...
1080: e8 cb ff ff ff    call 1050 <rand@plt>
1085: 48 83 c3 04       add  rbx,0x4
1089: 89 43 fc          mov  DWORD PTR [rbx-0x4],eax
108c: 48 39 eb          cmp  rbx,rbp
108f: 75 ef            jne  1080 <main+0x20>
1091: 4c 89 e7          mov  rdi,r12
1094: be 40 42 0f 00    mov  esi,0xf4240
1099: e8 52 02 00 00    call 12f0 <g>
...
00000000000012f0 <g>:
...
1335: e8 86 fe ff ff    call 11c0 <f>
...
$
```

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Achatamento do fluxo de execução (*control-flow flattening*): substitui todas as estruturas de controle por uma única sentença de seleção (*dispatcher*)

```
1 // E/S padrão
2 #include <stdio.h>
3 // Tipos inteiros padronizados
4 #include <stdint.h>
5 uint32_t f1(uint32_t n) {
6     uint32_t i, r = 1;
7     for(i = 2; i <= n; i++)
8         r = r * i;
9     return r;
10 }
... ..
```

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Achatamento do fluxo de execução (*control-flow flattening*): substitui todas as estruturas de controle por uma única sentença de seleção (*dispatcher*)

```
1 // E/S padrão
2 #include <stdio.h>
3 ...
10 uint32_t f2(uint32_t n) {
11     uint32_t i = 0, r, state = 1;
12     while(state != 0) {
13         ...
20     }
21     return r;
22 }
```

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Achatamento do fluxo de execução (*control-flow flattening*): substitui todas as estruturas de controle por uma única sentença de seleção (*dispatcher*)

```
1 // E/S padrão
2 #include <stdio.h>
3 ...
4
5     switch(state) {
6         case 1: r = 1; state = 2; break;
7         case 2: i = 2; state = 3; break;
8         case 3: if(i <= n) state = 4; else state =
9                 0; break;
10        case 4: r = r * i; state = 5; break;
11        case 5: i++; state = 3; break;
12    }
13
14    ...
```

Ofuscação

- ▶ Técnicas baseadas em controle de fluxo
 - ▶ Achatamento do fluxo de execução (*control-flow flattening*): substitui todas as estruturas de controle por uma única sentença de seleção (*dispatcher*)

```
0000000000001180 <f1>:
1180: 83 ff 01          cmp     edi,0x1
1183: 0f 86 cf 00 00 00 jbe     1258 <f1+0xd0>
1189: 8d 47 fe          lea     eax,[rdi-0x2]
118c: 83 f8 08          cmp     eax,0x8
118f: 0f 86 cb 00 00 00 jbe     1260 <f1+0xe0>
1195: 83 ff ff          cmp     edi,0xffffffff
1198: 0f 84 c2 00 00 00 je      1260 <f1+0xe0>
119e: 8d 4f ff          lea     ecx,[rdi-0x1]
...
1258: b8 01 00 00 00    mov     eax,0x1
125d: c3               ret
125e: 66 90           xchg    ax,ax
1260: b8 01 00 00 00    mov     eax,0x1
1265: ba 02 00 00 00    mov     edx,0x2
126a: 66 0f 1f 44 00 00 nop     WORD PTR [rax+
    ax*1+0x0]
1270: 0f af c2         imul    eax,edx
1273: 83 c2 01         add     edx,0x1
1276: 39 d7           cmp     edi,edx
1278: 73 f6           jae     1270 <f1+0xf0>
127a: c3             ret
127b: 0f 1f 44 00 00    nop     DWORD PTR [rax+
    rax*1+0x0]
```

```
0000000000001280 <f2>:
1280: b8 01 00 00 00    mov     eax,0x1
1285: ba 02 00 00 00    mov     edx,0x2
128a: 66 0f 1f 44 00 00 nop     WORD PTR [rax+r
    ax*1+0x0]
1290: 39 d7           cmp     edi,edx
1292: 73 04           jae     1298 <f2+0x18>
1294: c3             ret
1295: 0f 1f 00         nop     DWORD PTR [rax]
1298: 0f af c2         imul    eax,edx
129b: 83 c2 01         add     edx,0x1
129e: eb f0           jmp     1290 <f2+0x10>
```

Ofuscação

- ▶ Técnicas baseadas em dados
 - ▶ Esquemas de codificação: processam os dados armazenados na memória para gerar os valores esperados (*constant unfolding*)

```
1 // Delta value
2 #define DELTA 0xEA51F0F0
3 //  $\sim(\sim x + 1) = x - 1$ 
4 #define decrement(x)\
5     (*(x)) = (-(*(x))); \
6     (*(x)) = (~(*(x)))
7 //  $-(-x - 1) = x + 1$ 
8 #define increment(x)\
9     (*(x)) = (~(*(x))); \
10    (*(x)) = (-(*(x)))
11 // Read value
12 #define read(x)\
13     ((x) + (DELTA))
14 // Write value
15 #define write(x)\
16     ((x) - (DELTA))
```

Ofuscação

- ▶ Técnicas baseadas em dados
 - ▶ Esquemas de codificação: processam os dados armazenados na memória para gerar os valores esperados (*constant unfolding*)

```
1 // Addition method 1
2 #define add1(z, x, y)\
3     (z) = read(x) + read(y);\
4     (z) = write(z)
5 // Addition method 2
6 #define add2(z, x, y)\
7     (z) = rand();\
8     while((z)--){\
9         increment(&x);\
10        decrement(&y);\
11    }\
12    add1(z, x, y)
```

Exercício

- ▶ Reproduza as técnicas de ofuscação de código baseadas em controle de fluxo e de dados
 - ▶ Verifique as técnicas de ofuscação por criptografia e geração de código em tempo de execução
 - ▶ Aplique os conceitos vistos para resolver o desafio proposto, sem utilizar técnicas invasivas, permitindo a execução do software sem modificações